

1 Solutions for Monday the 11th, 2026

1.1 Task 1

Notation

- c_{ij} : cost of arc (i, j)
- s, t : source and destination for the shortest path computation
- A : arcs that can be used

$$\min \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}$$

s.t.

$$\begin{aligned} \sum_{j=1}^N x_{sj} - \sum_{j=1}^N x_{js} &= 1 \\ \sum_{j=1}^N x_{kj} - \sum_{j=1}^N x_{jk} &= 0 & \forall k = 1, \dots, N, k \neq s, k \neq t \\ \sum_{j=1}^N x_{tj} - \sum_{j=1}^N x_{jt} &= -1 \\ x_{ij} &= 0 & \forall i = 1, \dots, N, j = 1, \dots, N, (i, j) \notin A \\ x_{ij} &\geq 0 \end{aligned}$$

1.1.1 Model using $x[i,j]$ variables, defined for all combinations of i and j and with variables corresponding to non-existent arcs fixed to 0:

```
using JuMP
using HiGHS
```

```
# -----
# Nodes (fixed order)
# 1: CPH (Copenhagen)
# 2: FRA (Frankfurt)
# 3: DXB (Dubai)
# 4: SIN (Singapore)
# 5: DOH (Doha)
# 6: SYD (Sydney)
# -----
nodes = ["CPH", "FRA", "DXB", "SIN", "DOH", "SYD"]
n = 6
```

```

S = 1 # CPH
T = 6 # SYD

# -----
# Cost matrix c[i,j]
# c[i,j] = cost in hundreds of euros
# c[i,j] = 0 ==> arc (i,j) does NOT exist
# -----
c = [
    0 2 6 0 0 0; # CPH
    0 0 5 2 5 0; # FRA
    0 0 0 5 2 0; # DXB
    0 0 0 0 0 5; # SIN
    0 0 0 6 0 10; # DOH
    0 0 0 0 0 0 # SYD
]

# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

# Flow variables on ALL pairs
@variable(model, x[1:n, 1:n] >= 0)

# Fix infeasible arcs to zero (where c[i,j] = 0)
for i in 1:n, j in 1:n
    if c[i,j] == 0
        fix(x[i,j], 0.0; force = true)
    end
end

# Objective: minimize total cost
@objective(model, Min,
    sum(c[i,j] * x[i,j] for i in 1:n, j in 1:n)
)

# -----
# Flow conservation constraints
# -----

# Source (Copenhagen): outflow - inflow = 1
@constraint(model,
    sum(x[S,j] for j in 1:n) - sum(x[i,S] for i in 1:n) == 1

```

```

)

# Destination (Sydney): inflow - outflow = 1
@constraint(model,
    sum(x[i,T] for i in 1:n) - sum(x[T,j] for j in 1:n) == 1
)

# Intermediate nodes: inflow = outflow
for v in 1:n
    if v != S && v != T
        @constraint(model,
            sum(x[i,v] for i in 1:n) == sum(x[v,j] for j in 1:n)
        )
    end
end

# -----
# Solve
# -----
optimize!(model)
println("Termination status: ", termination_status(model))
if termination_status(model) == MOI.OPTIMAL
    println("Termination status: ", termination_status(model))
    println("Optimal objective (hundreds of euro): ", objective_value(model))
    println("Approximate cost (euro): ", 100 * objective_value(model))

    println("\nChosen arcs:")
    for i in 1:n, j in 1:n
        if value(x[i,j]) > 1e-6
            println(" ", nodes[i], " -> ", nodes[j],
                " (x = ", value(x[i,j]), ", c = ", c[i,j], ")")
        end
    end
else
    println("Model not optimal (maybe infeasible).")
end

```

1.1.2 Model using $x[a \text{ in arcs}]$ variables.

using JuMP, HiGHS

$m = 6$ # Number of nodes

Node indices

1: CPH, 2: FRA, 3: DXB, 4: SIN, 5: DOH, 6: SYD

```

S = 1 # Copenhagen
T = 6 # Sydney

# Directed arcs
arcs = [
    (1,2), # CPH -> FRA
    (1,3), # CPH -> DXB
    (2,3), # FRA -> DXB
    (2,4), # FRA -> SIN
    (2,5), # FRA -> DOH
    (3,4), # DXB -> SIN
    (3,5), # DXB -> DOH
    (5,4), # DOH -> SIN
    (5,6), # DOH -> SYD
    (4,6)  # SIN -> SYD
]

# -----
# Arc costs (hundreds of euros)
# -----
c = Dict{Tuple{Int,Int}, Int}()

c[(1,2)] = 2 # CPH -> FRA
c[(1,3)] = 6 # CPH -> DXB

c[(2,3)] = 5 # FRA -> DXB
c[(2,4)] = 2 # FRA -> SIN
c[(2,5)] = 5 # FRA -> DOH

c[(3,4)] = 5 # DXB -> SIN
c[(3,5)] = 2 # DXB -> DOH

c[(5,4)] = 6 # DOH -> SIN
c[(5,6)] = 10 # DOH -> SYD

c[(4,6)] = 5 # SIN -> SYD

# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, x[a in arcs] >= 0)

@objective(model, Min, sum(c[a] * x[a] for a in arcs))

```

```

# 1) Source (Copenhagen): outflow - inflow = 1
@constraint(model,
    sum(x[a] for a in arcs if a[1] == S) -
    sum(x[a] for a in arcs if a[2] == S) == 1
)

# 2) Destination (Sydney): inflow - outflow = 1
@constraint(model,
    sum(x[a] for a in arcs if a[2] == T) -
    sum(x[a] for a in arcs if a[1] == T) == 1
)

# 3) Intermediate nodes: inflow = outflow
@constraint(model, [k in 1:m; k != S && k != T],
    sum(x[a] for a in arcs if a[2] == k) ==
    sum(x[a] for a in arcs if a[1] == k)
)

optimize!(model)

println("Termination status: ", termination_status(model))

if termination_status(model) == MOI.OPTIMAL
    println("Objective value (hundreds of euro): ", objective_value(model))
    println("Approximate cost (euro): ", 100 * objective_value(model))
    println("\nFlows on arcs:")
    for a in arcs
        println("    Flow on arc $a is ", value(x[a]))
    end
else
    println("Model not optimal (maybe infeasible).")
end

```

1.1.3 pros and cons of the two implementation approaches

- Models using $x[i,j]$ variables are typically the easiest to write. For very large graphs this is however an inefficient way of declaring the model. If we have a graph with 10,000 nodes we are going to create 100,000,000 variables and potentially fix most of them to zero.
- The $x[a \text{ in arcs}]$ variables allow us to create just the variables we need and for very large, sparse graphs this is much more efficient.

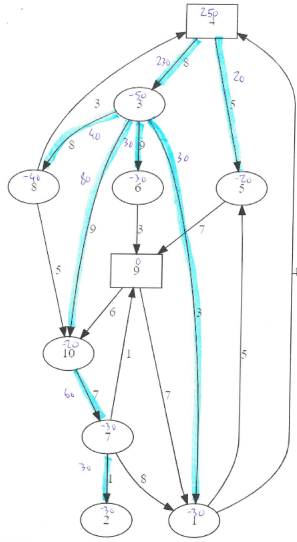
2 Task 2

2.1 Question A

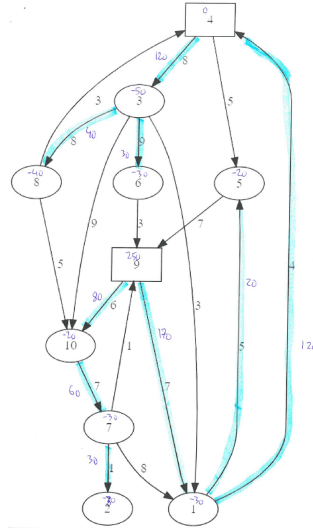
We evaluate the tree options by solving a minimum cost flow problem for each option.

- For the first option: “Place all supply (all 250 units) in node 4.” We setup a minimum cost flow problem as shown in figure 1.a with a supply of 250 in node 4 and zero supply in node 9. The figure also show how goods are transported along the arcs (handwritten numbers next to arcs show how many units are send through the arc). The objective of the minimum cost flow problem is 3790 which leads to a total cost of $3790+1000=\$4790$.
- The setup for the second option is similar and is show in figure 1.b. The objective of the minimum cost flow problem is 4250 which leads to a total cost of $4250+1000=\$5250$.
- For the third option we put a supply of 125 in both node 4 and node 9. The objective of the minimum cost flow problem is now 2895 which leads to a total cost of $2895+2000=\$4895$.

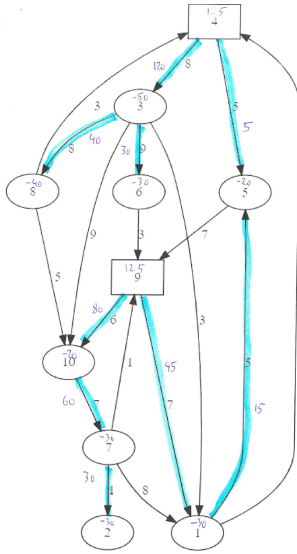
We see that the first option is the cheapest overall. The third option has very low transportation cost, but when including the cost of opening warehouses it becomes slightly more expensive compared to the first option.



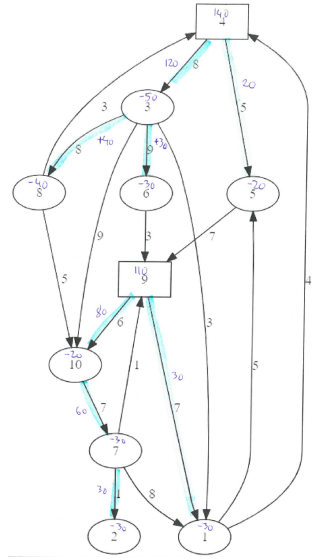
a) Warehouse is placed in node 4. Transport cost is 3790. Total cost is \$4790.



b) Warehouse is placed in node 9. Transport cost is 4250. Total cost is \$5250.



c) Warehouse is placed in node 4 and 9 (125 units in each node). Transport cost is 2895. Total cost is \$4895.



d) Warehouse is placed in node 4 and 9, supply distributed optimally (140 units in node 4 and 110 units in node 9). Transport cost is 2790. Total cost is \$4790.

Figure 1: The 4 different solution in task 3

```

# Task 2A
using JuMP, HiGHS

m = 10 # Number of nodes

# Directed arcs
arcs = [
    (1,4), (1,5),
    (3,1), (3,6), (3,8), (3,10),
    (4,3), (4,5),
    (5,9),
    (6,9),
    (7,1), (7,2), (7,9),
    (8,4), (8,10),
    (9,1), (9,10),
    (10,7)
]

# -----
# Arc costs (hundreds of euros)
# -----
c = Dict{Tuple{Int,Int}, Int}()

c[(1,4)] = 4
c[(1,5)] = 5
c[(3,1)] = 3
c[(3,6)] = 9
c[(3,8)] = 8
c[(3,10)] = 9
c[(4,3)] = 8
c[(4,5)] = 5
c[(5,9)] = 7
c[(6,9)] = 3
c[(7,1)] = 8
c[(7,2)] = 1
c[(7,9)] = 1
c[(8,4)] = 3
c[(8,10)] = 5
c[(9,1)] = 7
c[(9,10)] = 6
c[(10,7)] = 7

# Option 1
demands = [-30, -30, -50, 250, -20, -30, -30, -40, 0, -20]
# Option 2

```



```

#demands = [-30, -30, -50, 0, -20, -30, -30, -40, 250, -20]
# Option 3
#demands = [-30, -30, -50, 125, -20, -30, -30, -40, 125, -20]

# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, x[a in arcs] >= 0)

@objective(model, Min, sum(c[a] * x[a] for a in arcs))

@constraint(model, [k in 1:m],
    sum(x[a] for a in arcs if a[1] == k) - sum(x[a] for a in arcs if a[2] == k)
)

optimize!(model)

println("Termination status: ", termination_status(model))

if termination_status(model) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(model))
    for a in arcs
        println("Flow on arc (", a[1], ",", a[2], ") is ", JuMP.value(x[a]))
    end
end
end

```

2.1.1 task 2B

There are several ways to solve this problem. If we want to stick to a pure “minimum cost flow model” we can do as follows:

- Place all supply (all 250 units) in node 4.
- Add a dummy arc from node 4 to node 9 with cost zero and no upper bound. This allow us to redistribute the supply in node 4 to node 9 “for free”. The flow on this arc will give us the starting supply in node 9. The starting supply in node 4 will be given by 250 minus the flow on the arc.

Solving this model show that we should have 140 units in node 4 and 110 in node 9. In this case the flow cost are \$2790. Since we have to add \$2000 due to opening two warehouses. We see that we get exactly the same cost as we got when just opening a warehouse in node 4.

Task 2B

```
using JuMP, HiGHS
```

```
m = 10 # Number of nodes
```

```
# Directed arcs
```

```
arcs = [  
    (1,4), (1,5),  
    (3,1), (3,6), (3,8), (3,10),  
    (4,3), (4,5), (4,9), # (4,9) is added here. Dummy arc.  
    (5,9),  
    (6,9),  
    (7,1), (7,2), (7,9),  
    (8,4), (8,10),  
    (9,1), (9,10),  
    (10,7)  
]
```

```
# -----  
# Arc costs (hundreds of euros)  
# -----
```

```
c = Dict{Tuple{Int,Int}, Int}()
```

```
c[(1,4)] = 4  
c[(1,5)] = 5  
c[(3,1)] = 3  
c[(3,6)] = 9  
c[(3,8)] = 8  
c[(3,10)] = 9  
c[(4,3)] = 8  
c[(4,5)] = 5  
c[(4,9)] = 0 # Dummy arc with zero cost  
c[(5,9)] = 7  
c[(6,9)] = 3  
c[(7,1)] = 8  
c[(7,2)] = 1  
c[(7,9)] = 1  
c[(8,4)] = 3  
c[(8,10)] = 5  
c[(9,1)] = 7  
c[(9,10)] = 6  
c[(10,7)] = 7
```

```
# Option 1
```

```
demands = [-30, -30, -50, 250, -20, -30, -30, -40, 0, -20]
```

```

# -----
# Model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, x[a in arcs] >= 0)

@objective(model, Min, sum(c[a] * x[a] for a in arcs))

@constraint(model, [k in 1:m],
    sum(x[a] for a in arcs if a[1] == k) - sum(x[a] for a in arcs if a[2] == k)
)

optimize!(model)

println("Termination status: ", termination_status(model))

if termination_status(model) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(model))
    for a in arcs
        println("Flow on arc (", a[1], ",", a[2], ") is ", JuMP.value.(x[a]))
    end
end
end

```

2.2 Task 3

To solve this problem we have to turn the information in the exercise text into a graph. Once the graph is setup we need to formulate the multi-commodity flow problem as a LP and solve it (we use the multi-commodity flow formulation from the lectures without any changes).

- The arcs in the graph come from the definition of the rotations
- The arc capacities also come from the definition of the rotations
- The arc cost are given by the cost matrix in the exercise (numbers have to divided by 100)
- The commodities that we should ship through the graph are given in the exercise text.

We formulate the following LP and solve it.

```

# Task 3
using JuMP, HiGHS

# =====

```

```

# Multi-Commodity Flow (MCF) LP for container shipping
# =====
# Ports (nodes) are indexed 1..m:
# 1 ROT (Rotterdam)      2 HAM (Hamburg)      3 FEL (Felixstowe)
# 4 ALG (Algeciras)      5 DXB (Jebel Ali)    6 SIN (Singapore)
# 7 SHA (Shanghai)      8 BUS (Busan)      9 LAX (Los Angeles)
# 10 PAN (Panama)       11 NYC (New York)
# =====

ports = [
    "ROT", "HAM", "FEL", "ALG", "DXB", "SIN", "SHA", "BUS", "LAX", "PAN", "NYC"
]
m = length(ports)

# -----
# arcs: defined by ship rotation legs (directed)
# -----

# Define arcs by rotation (as directed legs)
arcs = [(1,2), (2,3), (3,4), (4,1),
        (4,5), (5,6), (6,4),
        (6,7), (7,8), (8,6),
        (7,9), (9,10), (10,11), (11,7),
        (1,11), (11,10), (10,1),
        (2,5), (5,7), (7,6), (6,2)]

# ROT->HAM->FEL->ALG->ROT
# ALG->DXB->SIN->ALG
# SIN->SHA->BUS->SIN
# SHA->LAX->PAN->NYC->SHA
# ROT->NYC->PAN->ROT
# HAM->DXB->SHA->SIN->HAM

# Capacity per directed arc a
cap = Dict{Tuple{Int,Int}, Float64}()
# ROT->HAM->FEL->ALG->ROT
cap[(1,2)] = 800.0
cap[(2,3)] = 800.0
cap[(3,4)] = 800.0
cap[(4,1)] = 800.0

# ALG->DXB->SIN->ALG
cap[(4,5)] = 1100.0
cap[(5,6)] = 1100.0
cap[(6,4)] = 1100.0

# SIN->SHA->BUS->SIN
cap[(6,7)] = 1400.0
cap[(7,8)] = 1400.0
cap[(8,6)] = 1400.0

# SHA->LAX->PAN->NYC->SHA
cap[(7,9)] = 1000.0
cap[(9,10)] = 1000.0
cap[(10,11)] = 1000.0
cap[(11,7)] = 1000.0

```

```

# ROT->NYC->PAN->ROT
cap[(1,11)] = 900.0
cap[(11,10)] = 900.0
cap[(10,1)] = 900.0

# HAM->DXB->SHA->SIN->HAM
cap[(2,5)] = 1200.0
cap[(5,7)] = 1200.0
cap[(7,6)] = 1200.0
cap[(6,2)] = 1200.0

# cost matrix:
c_matrix =
[0      4.5      3      22      65      105      195      200      160      88      62
4.5      0      5      24      66      107      197      202      162      90      64
3      5      0      21      64      104      194      199      158      86      61
22      24      21      0      52      85      165      170      150      82      65
65      66      64      52      0      58      102      108      135      150      110
105      107      104      85      58      0      38      46      141      167      155
195      197      194      165      102      38      0      9      104      125      190
200      202      199      170      108      46      9      0      98      120      185
160      162      158      150      135      141      104      98      0      48      63
88      90      86      82      150      167      125      120      48      0      35
62      64      61      65      110      155      190      185      63      35      0]

# -----
# Demands (commodities): (origin, destination, volume TEU/week)
# -----
demands = [
  (2, 7, 480.0), # k1: Hamburg -> Shanghai
  (1, 6, 400.0), # k2: Rotterdam -> Singapore
  (3, 5, 320.0), # k3: Felixstowe -> Jebel Ali
  (7, 11, 560.0), # k4: Shanghai -> New York
  (6, 10, 360.0), # k5: Singapore -> Panama
  (1, 11, 440.0), # k6: Rotterdam -> New York
  (2, 9, 520.0), # k7: Hamburg -> Los Angeles
  (8, 4, 300.0) # k8: Busan -> Algeciras
]
K = length(demands)

# -----
# Build LP model
# -----
model = Model(HiGHS.Optimizer)
set_silent(model)

# Flow variables: f[k, a] = TEU of commodity k shipped on arc a
@variable(model, f[a in arcs, 1:K] >= 0)

# Objective: minimize total cost

```

```

@objective(model, Min, sum(c_matrix[a[1], a[2]] * f[a,k] for k in 1:K, a in arcs))

# Capacity constraints: total flow on arc <= capacity
@constraint(model, [a in arcs], sum(f[a,k] for k in 1:K) <= cap[a])

# Flow conservation for each commodity
for k in 1:K
    o, d, q = demands[k]
    @constraint(model, [n in 1:m; n != o && n != d],
        sum(f[a,k] for a in arcs if a[2] == n) ==
        sum(f[a,k] for a in arcs if a[1] == n))
    @constraint(model, sum(f[a,k] for a in arcs if a[1] == o)
        - sum(f[a,k] for a in arcs if a[2] == o) == q)
    @constraint(model, sum(f[a,k] for a in arcs if a[1] == d)
        - sum(f[a,k] for a in arcs if a[2] == d) == -q)
end

optimize!(model)

println("Termination status: ", termination_status(model))
if termination_status(model) == MOI.OPTIMAL
    println("Objective value: ", objective_value(model))

    # Print arc utilizations
    println("\nArc utilizations (total flow / capacity):")
    for a in sort(arcs)
        used = sum(value(f[a,k]) for k in 1:K)
        if used > 1e-6
            println("  $(ports[a[1]]) -> $(ports[a[2]]): ",
                round(used, digits=2), " / ", cap[a])
        end
    end

    # Print per-commodity flows (only positive arcs)
    println("\nPer-commodity routed arcs:")
    for k in 1:K
        ok, dk, qk = demands[k]
        println("Commodity $k: $(ports[ok]) -> $(ports[dk]) (", qk, " TEU)")
        for a in sort(arcs)
            val = value(f[a,k])
            if val > 1e-6
                println("  $(ports[a[1]]) -> $(ports[a[2]]): ", round(val, digits=2))
            end
        end
    end
end
else
    println("Model not optimal (maybe infeasible).")
end
end

```

Output

```
Termination status: OPTIMAL
Objective value: 541380.0
Arc utilizations (total flow / capacity):
  ROT -> HAM: 400.0 / 800.0
  ROT -> NYC: 880.0 / 900.0
  HAM -> FEL: 200.0 / 800.0
  HAM -> DXB: 1200.0 / 1200.0
  FEL -> ALG: 520.0 / 800.0
  ALG -> ROT: 440.0 / 800.0
  ALG -> DXB: 520.0 / 1100.0
  DXB -> SIN: 1100.0 / 1100.0
  DXB -> SHA: 300.0 / 1200.0
  SIN -> ALG: 740.0 / 1100.0
  SIN -> SHA: 700.0 / 1400.0
  SHA -> SIN: 80.0 / 1200.0
  SHA -> LAX: 1000.0 / 1000.0
  BUS -> SIN: 300.0 / 1400.0
  LAX -> PAN: 480.0 / 1000.0
  PAN -> NYC: 480.0 / 1000.0
  NYC -> PAN: 360.0 / 900.0
Per-commodity routed arcs:
Commodity 1: HAM -> SHA (480.0 TEU)
  HAM -> FEL: 200.0
  HAM -> DXB: 280.0
  FEL -> ALG: 200.0
  ALG -> DXB: 200.0
  DXB -> SIN: 180.0
  DXB -> SHA: 300.0
  SIN -> SHA: 180.0
Commodity 2: ROT -> SIN (400.0 TEU)
  ROT -> HAM: 400.0
  HAM -> DXB: 400.0
  DXB -> SIN: 400.0
Commodity 3: FEL -> DXB (320.0 TEU)
  FEL -> ALG: 320.0
  ALG -> DXB: 320.0
Commodity 4: SHA -> NYC (560.0 TEU)
  ROT -> NYC: 80.0
  ALG -> ROT: 80.0
  SIN -> ALG: 80.0
  SHA -> SIN: 80.0
  SHA -> LAX: 480.0
  LAX -> PAN: 480.0
  PAN -> NYC: 480.0
Commodity 5: SIN -> PAN (360.0 TEU)
  ROT -> NYC: 360.0
  ALG -> ROT: 360.0
  SIN -> ALG: 360.0
```

```

    NYC -> PAN: 360.0
Commodity 6: ROT -> NYC  (440.0 TEU)
    ROT -> NYC: 440.0
Commodity 7: HAM -> LAX  (520.0 TEU)
    HAM -> DXB: 520.0
    DXB -> SIN: 520.0
    SIN -> SHA: 520.0
    SHA -> LAX: 520.0
Commodity 8: BUS -> ALG  (300.0 TEU)
    SIN -> ALG: 300.0
    BUS -> SIN: 300.0

```

2.3 Task 4A

Notation

- N : number of nodes
- K : number of commodities/demands
- D_k amount of containers on demand k
- b_{ik} . supply of commodity k in node i (D_k for the source of commodity k , $-D_k$ for the destination and 0 otherwise)
- d_{ij} : distance from node i to j in km.

Variables

- $x_{ij} \in \{0, 1\}$: Do we open an arc (1) or not (0)
- $f_{ijk} \geq 0$: Flow of commodity k over arc (i, j) . Unit: TEU

$$\min \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^K 0.01 d_{ij} f_{ijk} + \sum_{i=1}^N \sum_{j=1}^N 10 d_{ij} x_{ij}$$

subject to

$$\sum_{j=1}^N f_{ijk} - \sum_{i=1}^N f_{jik} = b_{ik} \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (1)$$

$$\sum_{k=1}^K f_{ijk} \leq 800 x_{ij} \quad \forall i = 1, \dots, N, j = 1, \dots, N \quad (2)$$

$$\sum_{i=1}^N x_{ii} = 0 \quad (3)$$

$$f_{ijk} \geq 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, k = 1, \dots, K \quad (4)$$

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, N, j = 1, \dots, N \quad (5)$$

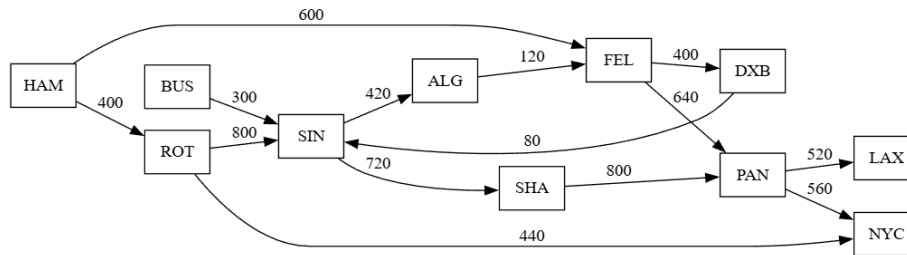
Equation (1) is the flow conservation constraint, (2) is the capacity constraint and (3) ensures that there are no self-loops.

Solution Objective value: 1207800

solve time: 20s

```
Commodity 1: from 2 to 7, demand 480.0
Flow on arc (1,6): 399.9999999999997
Flow on arc (2,1): 399.9999999999992
Flow on arc (2,3): 80.000000000000148
Flow on arc (3,5): 79.99999999999973
Flow on arc (5,6): 79.99999999999973
Flow on arc (6,7): 480.0
Commodity 2: from 1 to 6, demand 400.0
Flow on arc (1,6): 399.99999999999966
Commodity 3: from 3 to 5, demand 320.0
Flow on arc (3,5): 319.99999999999983
Commodity 4: from 7 to 11, demand 560.0
Flow on arc (7,10): 560.0
Flow on arc (10,11): 560.0
Commodity 5: from 6 to 10, demand 360.0
Flow on arc (3,10): 120.000000000000044
Flow on arc (4,3): 120.000000000000036
Flow on arc (6,4): 120.0
Flow on arc (6,7): 240.0
Flow on arc (7,10): 240.0
Commodity 6: from 1 to 11, demand 440.0
Flow on arc (1,11): 439.99999999999994
Commodity 7: from 2 to 9, demand 520.0
Flow on arc (2,3): 520.000000000000023
Flow on arc (3,10): 520.000000000000026
Flow on arc (10,9): 520.0
Commodity 8: from 8 to 4, demand 300.0
Flow on arc (6,4): 299.99999999999981
Flow on arc (8,6): 300.0
Arc utilizations (total flow / capacity):
ROT -> SIN: 800.0 / 800.0
ROT -> NYC: 440.0 / 800.0
HAM -> ROT: 400.0 / 800.0
HAM -> FEL: 600.0 / 800.0
FEL -> DXB: 400.0 / 800.0
FEL -> PAN: 640.0 / 800.0
ALG -> FEL: 120.0 / 800.0
DXB -> SIN: 80.0 / 800.0
SIN -> ALG: 420.0 / 800.0
SIN -> SHA: 720.0 / 800.0
SHA -> PAN: 800.0 / 800.0
BUS -> SIN: 300.0 / 800.0
PAN -> LAX: 520.0 / 800.0
PAN -> NYC: 560.0 / 800.0
Used arcs:
Arc (1,6) is used.
Arc (1,11) is used.
```

Arc (2,1) is used.
 Arc (2,3) is used.
 Arc (3,5) is used.
 Arc (3,10) is used.
 Arc (4,3) is used.
 Arc (5,6) is used.
 Arc (6,4) is used.
 Arc (6,7) is used.
 Arc (7,10) is used.
 Arc (8,6) is used.
 Arc (10,9) is used.
 Arc (10,11) is used.



Julia listing (only included for 4A, the code for the other tasks are similar:

```

# task 4A
using JuMP, HiGHS

c_matrix =
[ 0  4.5  3  22  65  105  195  200  160  88  62
  4.5 0  5  24  66  107  197  202  162  90  64
  3  5  0  21  64  104  194  199  158  86  61
  22  24  21  0  52  85  165  170  150  82  65
  65  66  64  52  0  58  102  108  135  150  110
  105  107  104  85  58  0  38  46  141  167  155
  195  197  194  165  102  38  0  9  104  125  190
  200  202  199  170  108  46  9  0  98  120  185
  160  162  158  150  135  141  104  98  0  48  63
  88  90  86  82  150  167  125  120  48  0  35
  62  64  61  65  110  155  190  185  63  35  0]

N = size(c_matrix, 1)

ports = [
  "ROT", "HAM", "FEL", "ALG", "DXB", "SIN", "SHA", "BUS", "LAX", "PAN", "NYC"
]

ship_capacity = 800.0 # TEU

# -----
# Demands (commodities): (origin, destination, volume TEU/week)
# -----
demands = [
  (2, 7, 480.0), # k1: Hamburg -> Shanghai
  (1, 6, 400.0), # k2: Rotterdam -> Singapore
  (3, 5, 320.0), # k3: Felixstowe -> Jebel Ali
  (7, 11, 560.0), # k4: Shanghai -> New York
  (6, 10, 360.0), # k5: Singapore -> Panama
  (1, 11, 440.0), # k6: Rotterdam -> New York
  (2, 9, 520.0), # k7: Hamburg -> Los Angeles
  (8, 4, 300.0) # k8: Busan -> Algeciras
]
K = length(demands)

model = Model(HiGHS.Optimizer)
#set_silent(model)

@variable(model, f[1:N, 1:N, 1:K] >= 0)
@variable(model, x[1:N, 1:N], Bin)
@objective(model, Min, sum(c_matrix[i,j] * f[i,j,k] for i in 1:N, j in 1:N, k in 1:K) + sum(c_matrix[i,j] * 1000 * x[i,j] for i in 1:N, j in 1:N))
# Flow conservation constraints
for k in 1:K
  o, d, q = demands[k]
  @constraint(model, [i in 1:N; i!=o && i!=d], sum(f[i,j,k] for j in 1:N) - sum(f[j,i,k] for j in 1:N) == 0)
end

```

```

@constraint(model, sum(f[o,j,k] for j in 1:N) - sum(f[j,o,k] for j in 1:N) == q)
@constraint(model, sum(f[d,i,k] for i in 1:N) - sum(f[i,d,k] for i in 1:N) == -q)
end
# Capacity constraints
@constraint(model, [i in 1:N, j in 1:N], sum(f[i,j,k] for k in 1:K) <= ship_capacity * x[i,j])

optimize!(model)

println("Termination status: ", termination_status(model))
if termination_status(model) == MOI.OPTIMAL
    for k in 1:K
        o, d, q = demands[k]
        println("Commodity $k: from $o to $d, demand $q")
        for i in 1:N, j in 1:N
            if value(f[i,j,k]) > 0.1
                println("    Flow on arc ($i,$j): ", value(f[i,j,k]))
            end
        end
    end

    # Print arc utilizations
    println("\nArc utilizations (total flow / capacity):")
    for i=1:N, j=1:N
        used = sum(value(f[i,j,k]) for k in 1:K)
        if used > 1e-6
            println("    $(ports[i]) -> $(ports[j]): ", round(used, digits=2), " / ", ship_capacity)
        end
    end

    println("Used arcs:")
    for i in 1:N, j in 1:N
        if value(x[i,j]) > 0.1
            println("    Arc ($i,$j) is used.")
        end
    end
end
end

```

2.4 Task 4B

Model:

$$\min \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^K 0.01 d_{ij} f_{ijk} + \sum_{i=1}^N \sum_{j=1}^N 10 d_{ij} x_{ij}$$

subject to

$$\sum_{j=1}^N f_{ijk} - \sum_{i=1}^N f_{jik} = b_{ik} \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (6)$$

$$\sum_{k=1}^K f_{ijk} \leq 800 x_{ij} \quad \forall i = 1, \dots, N, j = 1, \dots, N \quad (7)$$

$$\sum_{i=1}^N x_{ii} = 0 \quad (8)$$

$$\sum_{j=1}^N x_{ij} - \sum_{i=1}^N x_{ji} = 0 \quad \forall i = 1, \dots, N \quad (9)$$

$$f_{ijk} \geq 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, k = 1, \dots, K \quad (10)$$

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, N, j = 1, \dots, N \quad (11)$$

We have added constraint (9). This ensures that the number of “x-arcs” that enter a node is equal to the number of “x-arcs” that leave the node

Solution Objective value: 1430540
solve time: 7s

```

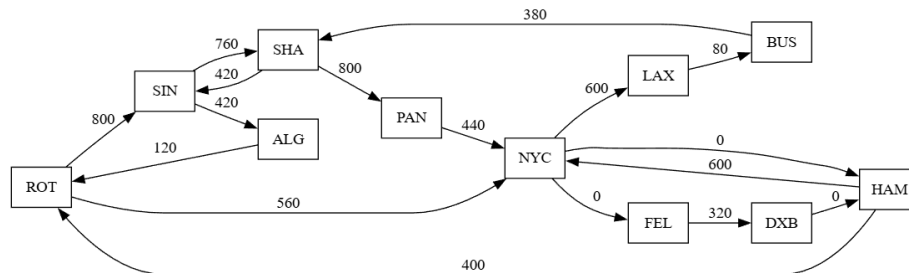
Commodity 1: from 2 to 7, demand 480.0
Flow on arc (1,6): 400.0000000000188
Flow on arc (2,1): 400.0000000000188
Flow on arc (2,11): 80.0000000000071
Flow on arc (6,7): 399.9999999999926
Flow on arc (8,7): 80.0000000000071
Flow on arc (9,8): 80.0000000000071
Flow on arc (11,9): 80.0000000000071
Commodity 2: from 1 to 6, demand 480.0
Flow on arc (1,6): 399.9999999999858
Commodity 3: from 3 to 5, demand 320.0
Flow on arc (3,5): 319.9999999999625
Commodity 4: from 7 to 11, demand 560.0
Flow on arc (1,11): 119.99999999997902
Flow on arc (4,1): 119.99999999997418

```

```

Flow on arc (6,4): 120.00000000000000304
Flow on arc (7,6): 120.00000000000000473
Flow on arc (7,10): 440.00000000000000006
Flow on arc (10,11): 440.00000000000000006
Commodity 5: from 6 to 10, demand 360.0
Flow on arc (6,7): 360.0000000000000011
Flow on arc (7,10): 360.0000000000000011
Commodity 6: from 1 to 11, demand 440.0
Flow on arc (1,11): 439.999999999999947
Commodity 7: from 2 to 9, demand 520.0
Flow on arc (2,11): 519.999999999999702
Flow on arc (11,9): 520.000000000000019
Commodity 8: from 8 to 4, demand 300.0
Flow on arc (6,4): 300.0000000000000006
Flow on arc (7,6): 300.0
Flow on arc (8,7): 299.999999999999994
Arc utilizations (total flow / capacity):
ROT -> SIN: 800.0 / 800.0
ROT -> NYC: 560.0 / 800.0
HAM -> ROT: 400.0 / 800.0
HAM -> NYC: 600.0 / 800.0
FEL -> DXB: 320.0 / 800.0
ALG -> ROT: 120.0 / 800.0
SIN -> ALG: 420.0 / 800.0
SIN -> SHA: 760.0 / 800.0
SHA -> SIN: 420.0 / 800.0
SHA -> PAN: 800.0 / 800.0
SHA -> NYC: 380.0 / 800.0
PAN -> NYC: 440.0 / 800.0
LAX -> BUS: 80.0 / 800.0
LAX -> NYC: 600.0 / 800.0
BUS -> LAX: 0.0 / 800.0
NYC -> HAM: 600.0 / 800.0
NYC -> FEL: 0.0 / 800.0
FEL -> DXB: 320.0 / 800.0
DXB -> HAM: 0.0 / 800.0
Used arcs:
Arc (1,6) is used.
Arc (1,11) is used.
Arc (2,1) is used.
Arc (2,11) is used.
Arc (3,5) is used.
Arc (4,1) is used.
Arc (5,2) is used.
Arc (6,4) is used.
Arc (6,7) is used.
Arc (7,6) is used.
Arc (7,10) is used.
Arc (8,7) is used.
Arc (9,8) is used.
Arc (10,11) is used.
Arc (11,2) is used.
Arc (11,3) is used.
Arc (11,9) is used.

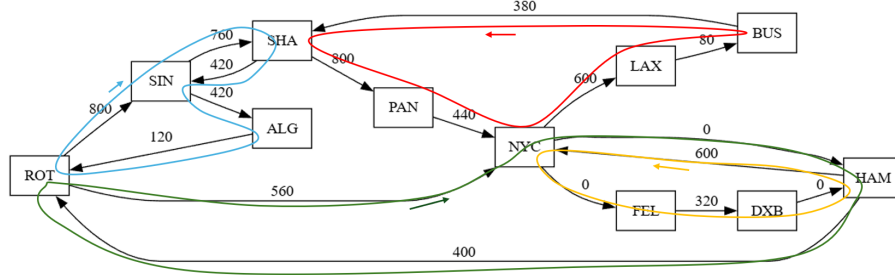
```



We see that we now have a nicer cyclic structure, in the next task we will see how to decompose the graph into cycles.

2.5 Task 4C

I would suggest forming 4 rotations as indicated on the figure below



2.6 Task 4D

New notation:

- S : number of ship types
- c_s cost per km for ship type s
- q_s capacity of ship type s

Changed variables:

- $x_{ijs} \in \{0, 1\}$: 1 if we open arc (i, j) using ship type s , 0 otherwise

Model:

$$\min \sum_{i=1}^N \sum_{j=1}^N \sum_{k=1}^K 0.01 d_{ij} f_{ijk} + \sum_{i=1}^N \sum_{j=1}^N \sum_{s=1}^S c_s d_{ij} x_{ijs}$$

subject to

$$\sum_{j=1}^N f_{ijk} - \sum_{i=1}^N f_{jik} = b_{ik} \quad \forall i = 1, \dots, N, k = 1, \dots, K \quad (12)$$

$$\sum_{k=1}^K f_{ijk} \leq \sum_{s=1}^S q_s x_{ijs} \quad \forall i = 1, \dots, N, j = 1, \dots, N \quad (13)$$

$$\sum_{i=1}^N x_{ii} = 0 \quad (14)$$

$$\sum_{j=1}^N x_{ijs} - \sum_{i=1}^N x_{jis} = 0 \quad \forall i = 1, \dots, N, s = 1, \dots, S \quad (15)$$

$$f_{ijk} \geq 0 \quad \forall i = 1, \dots, N, j = 1, \dots, N, k = 1, \dots, K \quad (16)$$

$$x_{ijs} \in \{0, 1\} \quad \forall i = 1, \dots, N, j = 1, \dots, N, s = 1, \dots, S \quad (17)$$

We have added a new index to the x variables to indicate the ship type. In the objective function we now multiply with the right “ship cost” per km depending on s . In (13) we use the right capacity depending on the ship type and in (15) we ensure that the node balance for ships is enforced for each ship type separately.

Solution

```

Objective value: 1329040
solution time 15.5 sec
Commodity 1: from 2 to 7, demand 480.0
Flow on arc (1,6): 400.00000000000006
Flow on arc (2,1): 400.00000000000006
Flow on arc (2,11): 79.999999999999994
Flow on arc (6,7): 400.00000000000006
Flow on arc (8,7): 79.999999999999994
Flow on arc (9,8): 79.999999999999994
Flow on arc (11,9): 79.999999999999994
Commodity 2: from 1 to 6, demand 400.0
Flow on arc (1,6): 400.0
Commodity 3: from 3 to 5, demand 320.0
Flow on arc (3,5): 320.000000000000038
Commodity 4: from 7 to 11, demand 560.0
Flow on arc (1,11): 120.000000000000007
Flow on arc (4,1): 120.000000000000007
Flow on arc (6,4): 120.000000000000011
Flow on arc (7,6): 120.00000000000001
Flow on arc (7,10): 439.999999999999994
Flow on arc (10,11): 439.999999999999994
Commodity 5: from 6 to 10, demand 360.0
Flow on arc (6,7): 360.0
Flow on arc (7,10): 360.0
Commodity 6: from 1 to 11, demand 440.0

```

```

Flow on arc (1,11): 440.00000000000006
Commodity 7: from 2 to 9, demand 520.0
Flow on arc (1,11): 199.99999999999994
Flow on arc (2,1): 199.99999999999994
Flow on arc (2,11): 320.00000000000006
Flow on arc (11,9): 520.0
Commodity 8: from 8 to 4, demand 300.0
Flow on arc (6,4): 300.0
Flow on arc (7,6): 300.0
Flow on arc (8,7): 300.0
Arc utilizations (total flow / capacity):
ROT -> SIN: 800.0
ROT -> NYC: 760.0
HAM -> ROT: 600.0
HAM -> NYC: 400.0
FEL -> DXB: 320.0
ALG -> ROT: 120.0
SIN -> ALG: 420.0
SIN -> SHA: 760.0
SHA -> SIN: 420.0
SHA -> PAN: 800.0
BUS -> SHA: 380.0
LAX -> BUS: 80.0
PAN -> NYC: 440.0
NYC -> LAX: 600.0
NYC -> FEL: 0.0
NYC -> HAM: 0.0
Used arcs:
Arc (1,6) with ship type 2 is used.
Arc (1,11) with ship type 2 is used.
Arc (2,1) with ship type 2 is used.
Arc (2,11) with ship type 1 is used.
Arc (3,5) with ship type 1 is used.
Arc (4,1) with ship type 2 is used.
Arc (5,2) with ship type 1 is used.
Arc (6,4) with ship type 2 is used.
Arc (6,7) with ship type 2 is used.
Arc (7,6) with ship type 2 is used.
Arc (7,10) with ship type 2 is used.
Arc (8,7) with ship type 2 is used.
Arc (9,8) with ship type 2 is used.
Arc (10,11) with ship type 2 is used.
Arc (11,2) with ship type 2 is used.
Arc (11,3) with ship type 1 is used.
Arc (11,9) with ship type 2 is used.

```

The solution looks as follows. (S1) / (S2) on the arc labels indicate the vessel type. We see that we mostly use ship vessel type 2 (the large one) but the NYC->FEL->DXB->HAM-NYC rotation has changed to the smaller vessels. This change has allowed the objective value to drop from 1,430,540 Euro to 1,329,040 Euro.

